

POLYMORPHISM IN DART

Introduction

In this section, you will learn about polymorphism in Dart programming language with the help of examples. Before learning about polymorphism in Dart, you should have a basic understanding of the [inheritance](#) in Dart.

Polymorphism In Dart

Poly means **many** and morph means **forms**. Polymorphism is the ability of an object to take on many forms. As humans, we have the ability to take on many forms. We can be a student, a teacher, a parent, a friend, and so on. Similarly, in object-oriented programming, polymorphism is the ability of an object to take on many forms.

Info

Note: In the real world, polymorphism is updating or modifying the feature, function, or implementation that already exists in the parent class.

Polymorphism By Method Overriding

Method overriding is a technique in which you can create a method in the child class that has the same name as the method in the parent class. The method in the child class overrides the method in the parent class.

Syntax

```
class ParentClass{  
  void functionName(){  
  }  
}  
class ChildClass extends ParentClass{  
  @override  
  void functionName(){
```



```
}  
}
```

Example 1: Polymorphism By Method Overriding In Dart

In this example below, there is a class named **Animal** with a method named **eat()**. The **eat()** method is overridden in the child class named **Dog**.

```
class Animal {  
  void eat() {  
    print("Animal is eating");  
  }  
}  
  
class Dog extends Animal {  
  @override  
  void eat() {  
    print("Dog is eating");  
  }  
}  
  
void main() {  
  Animal animal = Animal();  
  animal.eat();  
  
  Dog dog = Dog();  
  dog.eat();  
}
```

[Show Output](#)

[Run Online](#)

Example 2: Polymorphism By Method Overriding In Dart

In this example below, there is a class named **Vehicle** with a method named **run()**. The **run()** method is overridden in the child class named **Bus**.

```
class Vehicle {
```

```
void run() {  
    print("Vehicle is running");  
}  
  
class Bus extends Vehicle {  
    @override  
    void run() {  
        print("Bus is running");  
    }  
}  
  
void main() {  
    Vehicle vehicle = Vehicle();  
    vehicle.run();  
  
    Bus bus = Bus();  
    bus.run();  
}
```

[Show Output](#)

Run Online

Info

Note: If you don't write **@override**, the program still runs. But, it is a good practice to write **@override**.

Example 3: Polymorphism By Method Overriding In Dart

In this example below, there is a class named **Car** with a method named **power()**. The **power()** method is overridden in two child classes named **Honda** and **Tesla**.

```
class Car{  
    void power(){  
        print("It runs on petrol.");  
    }  
}  
  
class Honda extends Car{  
  
}  
  
class Tesla extends Car{  
    @override
```



```
void power(){
    print("It runs on electricity.");
}

void main(){
    Honda honda=Honda();
    Tesla tesla=Tesla();

    honda.power();
    tesla.power();
}
```

[Show Output](#)

Run Online

Example 4: Polymorphism By Method Overriding In Dart

In this example below, there is a class named **Employee** with a method named **salary()**. The **salary()** method is overridden in two child classes named **Manager** and **Developer**.

```
class Employee{
    void salary(){
        print("Employee salary is \">$1000.");
    }
}

class Manager extends Employee{
    @override
    void salary(){
        print("Manager salary is \ $2000.");
    }
}

class Developer extends Employee{
    @override
    void salary(){
        print("Developer salary is \ $3000.");
    }
}

void main(){
    Manager manager=Manager();
    Developer developer=Developer();

    manager.salary();
    developer.salary();
}
```

```
}
```

[Show Output](#)

[Run Online](#)

Advantage Of Polymorphism In Dart

- Subclasses can override the behavior of the parent class.
- It allows us to write code that is more flexible and reusable.

Video

Watch our video on polymorphism in Dart.